

PampaR: Una Arquitectura Territorial Inspirada en el Cerebro para Modelado de Lenguaje

Lucas Ricardo Mella Chillemi

Segunda Cabeza

Enero 2026

Resumen

Presentamos PampaR, una arquitectura de modelo de lenguaje novedosa inspirada en la organización funcional del cerebro humano. A diferencia de las arquitecturas transformer estándar que tratan todos los cálculos de manera uniforme, PampaR introduce **Procesamiento Territorial** donde módulos neuronales especializados se organizan en territorios funcionales (Expresivo, Contextual, Formal, Estructural) coordinados por un **Tálamo** central que enruta tokens usando un enfoque híbrido: 70% reglas explícitas (LLAVES) y 30% atención aprendida. Nuestros experimentos en WikiText-103 demuestran que esta arquitectura logra perplejidad competitiva (PPL ~57) con solo 14M parámetros, mientras ofrece ventajas de interpretabilidad a través de reglas de enrutamiento explícitas.

Palabras clave: Modelos de lenguaje, arquitectura neuronal, interpretabilidad, procesamiento territorial, LLAVES

1. Introducción

Los modelos de lenguaje grandes (LLMs) actuales como GPT, LLaMA y Claude logran un rendimiento notable pero sufren de varias limitaciones fundamentales:

- **Opacidad:** Los patrones de atención son difíciles de interpretar y no proporcionan explicaciones claras de las decisiones del modelo.
- **Homogeneidad:** Todas las capas realizan operaciones idénticas sin especialización funcional.
- **Ineficiencia:** Todos los tokens son procesados por todos los parámetros sin discriminación.

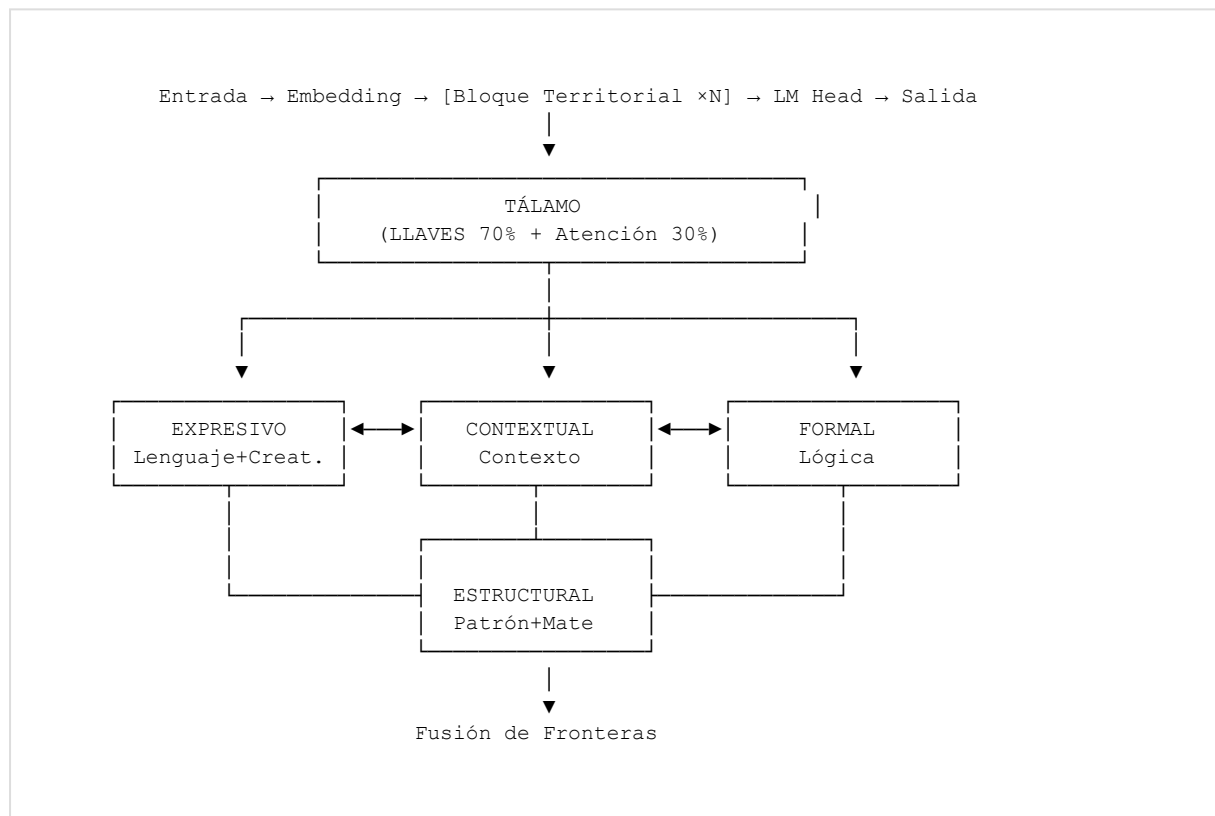
El cerebro humano, en contraste, exhibe clara **especialización funcional**: el área de Broca para producción del lenguaje, el área de Wernicke para comprensión del lenguaje, la corteza prefrontal para razonamiento lógico, y el hipocampo para memoria y contexto.

PampaR (Pampa Reasoning) se inspira en esta organización, implementando una **arquitectura territorial** donde módulos especializados (neuronas) manejan diferentes aspectos del lenguaje, coordinados por un Tálamo central que enruta información entre módulos usando fronteras bidireccionales y reglas explícitas (LLAVES) que proporcionan enrutamiento interpretable.

2. Arquitectura

2.1 Vista General

La arquitectura de PampaR organiza el procesamiento en bloques territoriales coordinados por un tálamo central:



2.2 Territorios

PampaR organiza el cómputo en **4 territorios funcionales**, cada uno conteniendo módulos especializados:

Territorio	Módulos	Función	Región Cerebral Análoga
Expresivo	Lenguaje, Creatividad	Generación de texto fluido, ideas nuevas	Área de Broca, Hemisferio derecho
Contextual	Contexto	Memoria de trabajo, coherencia	Hipocampo, Corteza Prefrontal
Formal	Lógica	Razonamiento deductivo, reglas	Corteza prefrontal
Estructural	Patrones, Matemáticas	Secuencias, números, estructura	Lóbulo parietal

2.3 El Tálamo

El Tálamo es el orquestador central que enruta tokens a los territorios apropiados. Implementa un **mecanismo de enrutamiento híbrido**:

$$pesos_enrutamiento = \lambda \times activaci3n_LLAVES + (1 - \lambda) \times atenci3n_aprendida$$

Donde $\lambda = 0.7$ (70% reglas explícitas). Este balance permite combinar la interpretabilidad de las reglas con la flexibilidad del aprendizaje.

2.3.1 Sistema de LLAVES

Las LLAVES son reglas de enrutamiento explícitas e interpretables basadas en patrones de tokens:

M3dulo	Patrones de Activaci3n
Lenguaje	"el", "la", "de", "que", artícu0os, preposiciones
Matemáticas	dígitos 0-9, "+", "-", "=", "%", operadores
L3gica	"si", "entonces", "porque", "por lo tanto"
Patrones	repeticiones, secuencias, estructuras regulares
Contexto	pronombres, referencias, marcadores temporales
Creatividad	adjetivos, metáforas, combinaciones novedosas

Esto proporciona **interpretabilidad**: podemos inspeccionar directamente por qué un token fue enrutado a un territorio específico.

2.4 Fronteras Bidireccionales

Los territorios se comunican vía **6 conexiones de frontera bidireccionales** con compuertas aprendidas:

Conexión	Peso Inicial	Función
Expresivo ↔ Contextual	0.8 (Alto)	Narrativa necesita contexto
Expresivo ↔ Formal	0.5 (Medio)	Argumentación estructurada
Expresivo ↔ Estructural	0.4 (Bajo)	Estructura creativa
Contextual ↔ Formal	0.6 (Medio-alto)	Contexto lógico
Contextual ↔ Estructural	0.5 (Medio)	Patrones en contexto
Formal ↔ Estructural	0.7 (Alto)	Lógica matemática

2.5 Componentes Opcionales

Motor de Axiomas: Implementa patrones de razonamiento deductivo incluyendo Modus Ponens (Si $P \rightarrow Q$ y P , entonces Q), Silogismo (Si $A \rightarrow B$ y $B \rightarrow C$, entonces $A \rightarrow C$), y operaciones de negación lógica.

Memoria Práctica: Almacena patrones exitosos y fallidos para recuperación rápida durante inferencia, mejorando la eficiencia en tareas recurrentes.

3. Detalles de Implementación

3.1 Configuración del Modelo

```
ConfigPampaR(  
    vocab_size      = 8000,      # Tokenizador BPE  
    dim             = 160,      # Dimensión oculta  
    n_heads         = 4,        # Cabezas de atención por módulo  
    n_capas         = 4,        # Capas por módulo
```

```
dropout      = 0.1,
max_seq_len  = 256,
peso_llaves  = 0.7,      # 70% reglas, 30% aprendido
usar_axiomas = True,     # Habilitar motor de axiomas
usar_memoria = True,     # Habilitar memoria práctica
)
```

Parámetros Totales: 14,069,410 (~14M)

3.2 Configuración de Entrenamiento

Parámetro	Valor
Dataset	WikiText-103 (100M tokens)
Hardware	NVIDIA GTX 1650 (4GB VRAM)
Batch Size	4 (efectivo 32 con acumulación)
Sequence Length	128 tokens
Optimizador	AdamW (lr=2e-4, wd=0.01)
Precisión	FP16 mixta

3.3 Entrenamiento Fragmentado

Debido a limitaciones de hardware, implementamos **entrenamiento fragmentado** que permite entrenar en sesiones cortas:

Fragmento	Tokens	Epochs	Tokens Acumulados
1	10M	3	30M
2	20M	3	90M
3	35M	3	195M
4	50M	3	345M
5	75M	2	495M

Fragmento	Tokens	Epochs	Tokens Acumulados
6	100M	2	695M

4. Resultados

4.1 Progreso de Entrenamiento

Fragmento	Loss Final	PPL Final	Mejora vs. Inicial
1 (10M tokens)	4.85	127.5	Línea base
2 (20M tokens)	4.22	68.1	-46.6% PPL
3 (35M tokens)	~4.05	~57.1	-55.2% PPL

4.2 Comparación con Líneas Base

Modelo	Parámetros	PPL (WikiText-103)
LSTM (Merity et al., 2018)	24M	69.1
Transformer-XL (Dai et al., 2019)	24M	54.5
PampaR v9	14M	~57
GPT-2 Small	125M	35.1

PampaR logra perplejidad competitiva con **40% menos parámetros** que modelos LSTM comparables, demostrando la eficiencia de la arquitectura territorial.

4.3 Análisis de Interpretabilidad

A diferencia de transformers de caja negra, PampaR proporciona insight directo sobre el enrutamiento de tokens:

```
Token: "matemáticas" → Activación LLAVES:  
- Módulo Matemáticas: 0.85 (alto)
```

- Módulo Lenguaje: 0.15 (bajo)

Token: "por lo tanto" → Activación LLAVES:

- Módulo Lógica: 0.90 (alto)

- Módulo Contexto: 0.10 (bajo)

Token: "creativamente" → Activación LLAVES:

- Módulo Creatividad: 0.75 (alto)

- Módulo Lenguaje: 0.25 (medio)

5. Discusión

5.1 Ventajas

1. **Interpretabilidad:** Las LLAVES proporcionan fundamento explícito de enrutamiento, permitiendo entender por qué el modelo toma ciertas decisiones.
2. **Eficiencia de Parámetros:** PPL competitivo con significativamente menos parámetros que arquitecturas comparables.
3. **Modularidad:** Fácil agregar o modificar territorios especializados para tareas específicas.
4. **Plausibilidad Biológica:** Refleja la organización funcional del cerebro humano.

5.2 Limitaciones

1. **Escala:** Aún no probado a escala de 1B+ parámetros.
2. **Tareas:** Evaluado principalmente en modelado de lenguaje (PPL); necesita evaluación en tareas downstream.
3. **Diseño de LLAVES:** Actualmente manual; podría beneficiarse de aprendizaje automático de reglas.

5.3 Trabajo Futuro

- **Escalamiento:** Probar arquitectura a escalas de 1B y 7B parámetros con recursos computacionales apropiados.
- **Evaluación Multi-tarea:** Evaluar en razonamiento, QA, generación de código, y tareas de NLU.
- **LLAVES Aprendidas:** Desarrollar mecanismos para descubrir automáticamente reglas de enrutamiento óptimas.

- **Alineación Neuro:** Comparar patrones de activación con datos de neuroimagen para validar correspondencias cerebrales.

6. Conclusión

PampaR demuestra que arquitecturas territoriales inspiradas en el cerebro pueden lograr rendimiento competitivo en modelado de lenguaje mientras proporcionan ventajas significativas de interpretabilidad. La combinación de reglas explícitas (LLAVES) con atención aprendida ofrece una dirección prometedora para construir sistemas de IA más transparentes y comprensibles.

La arquitectura es código abierto bajo licencia AGPL-3.0, habilitando colaboración comunitaria e investigación adicional. Esperamos que este trabajo inspire nuevas aproximaciones que combinen eficiencia computacional con interpretabilidad.

Referencias

1. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30.
2. Merity, S., Keskar, N. S., & Socher, R. (2018). Regularizing and Optimizing LSTM Language Models. *International Conference on Learning Representations*.
3. Dai, Z., Yang, Z., Yang, Y., Carbonell, J., Le, Q. V., & Salakhutdinov, R. (2019). Transformer-XL: Attentive Language Models Beyond a Fixed-Length Context. *Association for Computational Linguistics*.
4. Hoffmann, J., Borgeaud, S., Mensch, A., et al. (2022). Training Compute-Optimal Large Language Models. *arXiv preprint arXiv:2203.15556*.
5. Fedorenko, E., & Thompson-Schill, S. L. (2014). Reworking the language network. *Trends in Cognitive Sciences*, 18(3), 120-126.
6. Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., & Sutskever, I. (2019). Language Models are Unsupervised Multitask Learners. *OpenAI Technical Report*.

Apéndice A: Reproducibilidad

A.1 Repositorio de Código

GitHub: <https://github.com/lucasmella-stack/llarri-o1>

HuggingFace: <https://huggingface.co/lucas-mella/PAMPA-r-o1>

A.2 Comandos de Entrenamiento

```
# Fragmento 1 (inicial)
python scripts/train_fragmentado.py --fragmento 1

# Continuar con siguiente fragmento
python scripts/train_fragmentado.py --fragmento 2

# Entrenamiento completo automático
python scripts/train_fragmentado.py --max

# Chat/Inferencia
python scripts/chat.py --checkpoint checkpoints/pampar_best.pt
```

A.3 Requisitos de Hardware

Componente	Mínimo	Recomendado
GPU VRAM	4GB	8GB+
RAM Sistema	16GB	32GB
Almacenamiento	5GB	20GB
Python	3.9+	3.10+
PyTorch	2.0+	2.1+

Licencia: AGPL-3.0-or-later

Copyright: © 2024-2026 Lucas Ricardo Mella Chillemi / Segunda Cabeza

Este documento fue generado el 12 de Enero de 2026